

PROJECT DOCUMENTATION

THROW

AWS DevOps Infrastructure

A Flutter delivery marketplace, deployed with a fully automated CI/CD pipeline on Amazon Web Services

TERRAFORM · CODEPIPELINE · CODEBUILD · S3 · CLOUDFRONT · SECRETS
MANAGER · CLOUDWATCH

● STATUS — ALL THREE CI/CD PIPELINES FULLY
OPERATIONAL

Table of Contents

A section-by-section guide to this document, from project overview through deployment and lessons learned.

Executive Summary	4
<i>What Was Built</i>	<i>4</i>
<i>Current Status</i>	<i>4</i>
Project Overview	6
2.1 The Application	6
2.2 Project Goals	6
System Architecture	8
3.1 Architecture Diagram	8
3.2 Design Decisions	8
3.2.1 Per-app CodeBuild Projects via for_each	8
3.2.2 No Firebase Secrets in Git	9
3.2.3 Private S3, Public CDN	9
3.2.4 Remote Terraform State	9
AWS Infrastructure	10
4.1 Repository Layout (throw-infra)	10
4.2 Identity & Access Management	10
4.3 Storage Layout	11
4.4 Content Delivery	12
4.5 Live Terraform Outputs	12
CI/CD Pipeline Detail	14
5.1 Pipeline Stages	14
5.2 CodeBuild Phases	14
<i>Install Phase</i>	<i>14</i>
<i>Pre-build Phase</i>	<i>15</i>
<i>Build Phase</i>	<i>15</i>
<i>Post-build Phase</i>	<i>15</i>
5.3 Secrets Injected at Build Time	15
5.4 Build History	15
Security Decisions	17
6.1 No Credentials in Source Control	17
6.2 Private S3, OAC-Gated Access	17
6.3 Least-Privilege IAM	17
6.4 Git Credential Hygiene	18

Incident Log — Build Debugging Timeline	19
<i>Incident 1 — CloudFront Returned Access Denied</i>	<i>19</i>
<i>Incident 2 — Downloaded File Was the Wrong Artifact</i>	<i>19</i>
<i>Incident 3 — Android SDK Not Found</i>	<i>19</i>
<i>Incident 4 — Missing firebase_options.dart</i>	<i>20</i>
<i>Incident 5 — Gradle/Kotlin Daemon Crash (JVM OOM)</i>	<i>20</i>
<i>Incident 6 — Git Push Authentication Failure</i>	<i>21</i>
7.1 Status Snapshot — Before and After	21
How to Deploy / Reproduce	24
8.1 Prerequisites	24
8.2 Step-by-Step Deployment	24
<i>Step 1 — Create the Remote State Bucket</i>	<i>24</i>
<i>Step 2 — Configure Variables</i>	<i>24</i>
<i>Step 3 — Initialize and Review the Plan</i>	<i>25</i>
<i>Step 4 — Apply</i>	<i>25</i>
<i>Step 5 — Authorize the GitHub Connection (Manual, One-Time, Console Only)</i>	<i>25</i>
<i>Step 6 — Populate Firebase Secrets</i>	<i>26</i>
<i>Step 7 — Populate the Release-Signing Keystore</i>	<i>26</i>
<i>Step 8 — Trigger the First Real Build</i>	<i>27</i>
<i>Step 9 — Watch the Build, and What to Check if It Fails</i>	<i>27</i>
<i>Step 10 — Retrieve the Download Links</i>	<i>28</i>
<i>Step 11 — Confirm Monitoring Is Live</i>	<i>28</i>
<i>Step 12 — Tear Down Between Sessions (Cost Control)</i>	<i>29</i>
Installing the Built APK	30
9.1 Direct Download on an Android Device	30
9.2 Sharing via WhatsApp	30
9.3 BlueStacks (Emulator)	30
Monitoring and Cost Control	31
10.1 CloudWatch	31
10.2 SNS Alerting	31
10.3 Billing Alarm	31
10.4 Estimated Monthly Cost	31
Repository Summary	32
Lessons Learned	33
Next Steps	34

Executive Summary

Throw is a three-app Flutter delivery marketplace consisting of a customer app, a courier (delivery) app, and an admin app. This project took the existing Flutter codebase and built a complete, production-style DevOps platform around it on AWS, using Terraform for infrastructure as code.

The objective was to demonstrate an end-to-end DevOps workflow: a developer pushes code to GitHub, and within minutes a signed, installable Android APK is automatically built and published to a global CDN — with no manual steps, no secrets in source control, and full monitoring in place.

What Was Built

- Three private GitHub repositories for the Flutter apps, cleaned of embedded Firebase credentials
- A fourth repository (throw-infra) containing 100% Terraform-defined AWS infrastructure
- Three independent CI/CD pipelines (one per app) using AWS CodePipeline and CodeBuild
- Secure secrets handling via AWS Secrets Manager — no Firebase keys or signing credentials ever committed to git
- A CloudFront CDN in front of a private S3 bucket, serving the built APKs worldwide
- CloudWatch monitoring, build-failure alarms, and a billing alarm to control AWS spend

Current Status

STATUS

All three pipelines — throw-throw_admin-main-dev, throw-throw_delivery-main-dev, and throw-throw_user-main-dev — are fully green end-to-end (Source → Build → Deploy), each producing a working, installable Android APK published automatically to CloudFront on every push to main. The out-of-memory build failure that initially blocked the user and delivery apps (Incident 5, Section 7) was resolved by upgrading the CodeBuild compute type, and verified successful on the next pipeline run (see Figure 1).

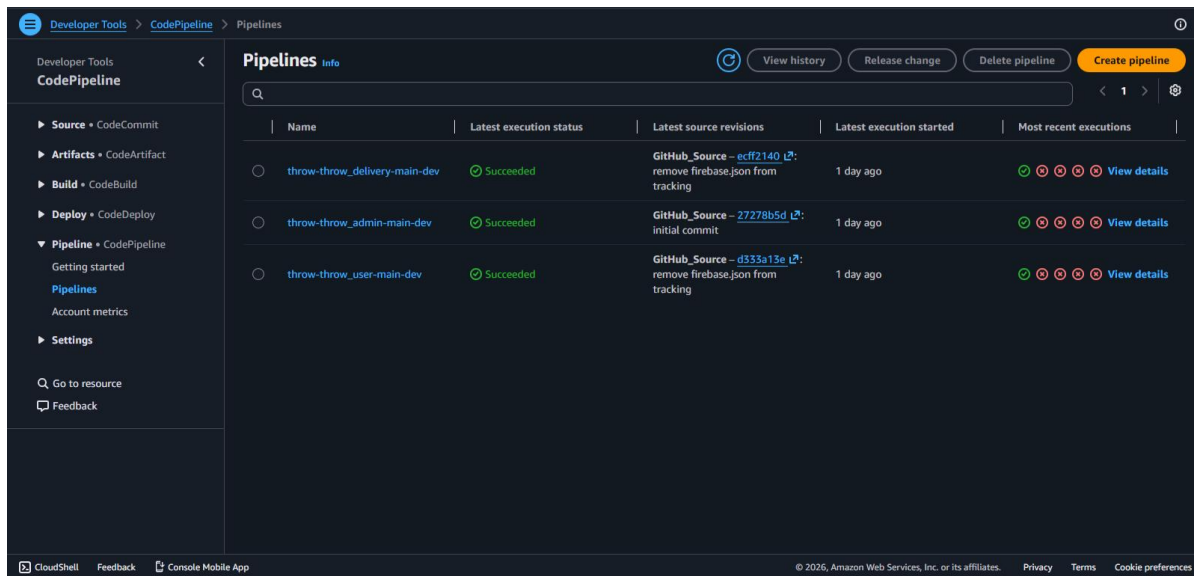


FIGURE 1 AWS CodePipeline console: all three pipelines (admin, delivery, user) showing Succeeded.

Project Overview

2.1 The Application

Throw is a delivery / courier marketplace. Customers post delivery requests, couriers bid on them in an auction-style flow, and an admin app manages the platform. The app uses Firebase for authentication, Firestore, and storage, and OpenStreetMap (OSM) for mapping and routing.

App	Repository	Role	Key Features
Customer app	throw_user-main	Places delivery requests	Auction bidding, real-time tracking, Firebase Auth
Courier app	throw_delivery-main	Fulfils deliveries	Browses auctions, bids, OTP delivery confirmation, GPS
Admin app	throw_admin-main	Platform administration	Pipeline fully operational; app UI at scaffold stage

2.2 Project Goals

- Understand and securely package a real, multi-app Flutter codebase
- Remove all hard-coded Firebase secrets from source control
- Provision 100% of AWS infrastructure as code, with no manual console configuration
- Build a fully automated CI/CD pipeline triggered by git push
- Distribute built APKs globally via CDN
- Add monitoring, alerting, and cost control
- Produce a clean, portfolio-ready GitHub presence across all repositories

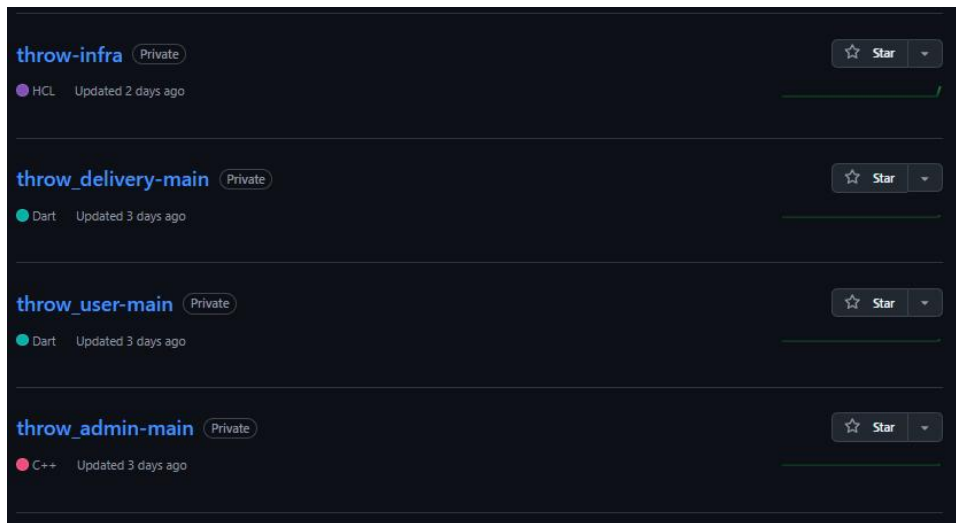


FIGURE 2 The four private GitHub repositories: three Flutter apps plus throw-infra (Terraform).

System Architecture

The platform follows a standard source-to-artifact CI/CD pattern. Each of the three Flutter apps has its own pipeline, but all three share the same underlying infrastructure pattern, the same S3 artifact bucket, and the same CloudFront distribution.

3.1 Architecture Diagram

```

GitHub (3 repos: throw_user-main, throw_delivery-main, throw_admin-main)
  |
  | git push to main
  v
AWS CodePipeline ----- IAM (least-privilege roles)
  |
  v
AWS CodeBuild
- installs Android SDK + cmdline-tools
- clones Flutter SDK (3.44.2)
- pulls Firebase secrets + signing keystore from Secrets Manager
- writes firebase_options.dart + google-services.json
- flutter pub get / build_runner
- flutter build apk --release
- uploads app-release.apk to S3
  |
  v
S3 (throw-artifacts-dev)
throw_user-main/app-release.apk
throw_delivery-main/app-release.apk
throw_admin-main/app-release.apk
  |
  v
CloudFront (Origin Access Control)
https://d1908c35bz5y71.cloudfront.net/<app>/app-release.apk
  |
  v
End user device (installs APK)

Monitoring:
CodeBuild logs --> CloudWatch Logs --> Alarms --> SNS --> email
Billing alarm (us-east-1) at a fixed monthly USD threshold

```

3.2 Design Decisions

3.2.1 Per-app CodeBuild Projects via for_each

Rather than writing three near-identical Terraform resources, the project uses a single `aws_codebuild_project` resource with `for_each = toset(var.apps)`, where `var.apps =`

["throw_user-main", "throw_delivery-main", "throw_admin-main"]. The same applies to CodePipeline and the Secrets Manager secrets. This keeps the codebase DRY and makes adding a fourth app trivial.

3.2.2 No Firebase Secrets in Git

Each app's google-services.json and firebase_options.dart were removed from the repositories and stored as encrypted values in AWS Secrets Manager. The buildspec injects them as environment variables and writes them to disk only inside the ephemeral CodeBuild container, immediately before the build step. The Android release-signing keystore and its password are stored the same way (throw/keystore and throw/keystore-password), so the APK is signed during the build without ever committing the keystore to source control.

3.2.3 Private S3, Public CDN

The artifact S3 bucket has no public access. CloudFront reaches it exclusively through an Origin Access Control (OAC), enforced by a bucket policy scoped to the specific CloudFront distribution ARN. Direct S3 URLs return Access Denied; only the CloudFront domain serves files.

3.2.4 Remote Terraform State

Terraform state is stored remotely in a dedicated, versioned S3 bucket (throw-terraform-state), separate from the artifact bucket, so that infrastructure changes are tracked and recoverable.

AWS Infrastructure

All infrastructure lives in the ap-south-1 (Mumbai) region, with the billing alarm in us-east-1 as required by AWS for that metric. Below is every AWS service used, its purpose in this project, and the relevant Free Tier allowance.

Service	Purpose in This Project	Free Tier
CodePipeline	Orchestrates source → build → deploy for each app	1 free active pipeline/month
CodeBuild	Installs Android SDK + Flutter, runs the release build	100 build-minutes/month free
S3 (artifacts)	Stores built, signed APKs and the pipeline build cache	5 GB for 12 months
S3 (pipeline cache)	Caches the Flutter pub-cache between builds	Included in S3 free tier
CloudFront	Global CDN serving APKs from the private S3 bucket	1 TB data transfer/month
Secrets Manager	Stores Firebase config and the release-signing keystore per app	~\$0.40/secret/month
IAM	Least-privilege roles for CodePipeline and CodeBuild	Always free
CloudWatch	Build logs, failure alarms, dashboard	10 alarms + 5 GB logs free
SNS	Email alerts for build failures and billing threshold	1,000 emails/month free
CodeStar Connections	Authenticates CodePipeline against GitHub	Free

4.1 Repository Layout (throw-infra)

```
throw-infra/
├── providers.tf           # AWS provider + remote state backend
├── variables.tf           # Input variable definitions
├── terraform.tfvars.example # Template (real values kept out of git)
├── s3.tf                  # Artifact bucket, cache bucket, log bucket
├── iam.tf                  # CodePipeline + CodeBuild IAM roles
├── codebuild.tf            # CodeBuild projects (for_each over 3 apps)
├── codepipeline.tf         # CodePipeline + GitHub connection
├── cloudfront.tf           # CloudFront distribution with OAC
├── secrets.tf              # Secrets Manager secret definitions
├── cloudwatch.tf           # Log groups, alarms, dashboard, SNS topic
└── outputs.tf              # CloudFront URLs, pipeline links, dashboard URL
```

4.2 Identity & Access Management

Five IAM roles exist in the account: three are AWS service-linked roles created automatically (Resource Explorer, Support, Trusted Advisor), and two are project-specific roles created by Terraform — `throw-codebuild-role-dev` and `throw-codepipeline-role-dev` — each scoped to the minimum permissions its service needs (see Section 6.3 for the exact policy).

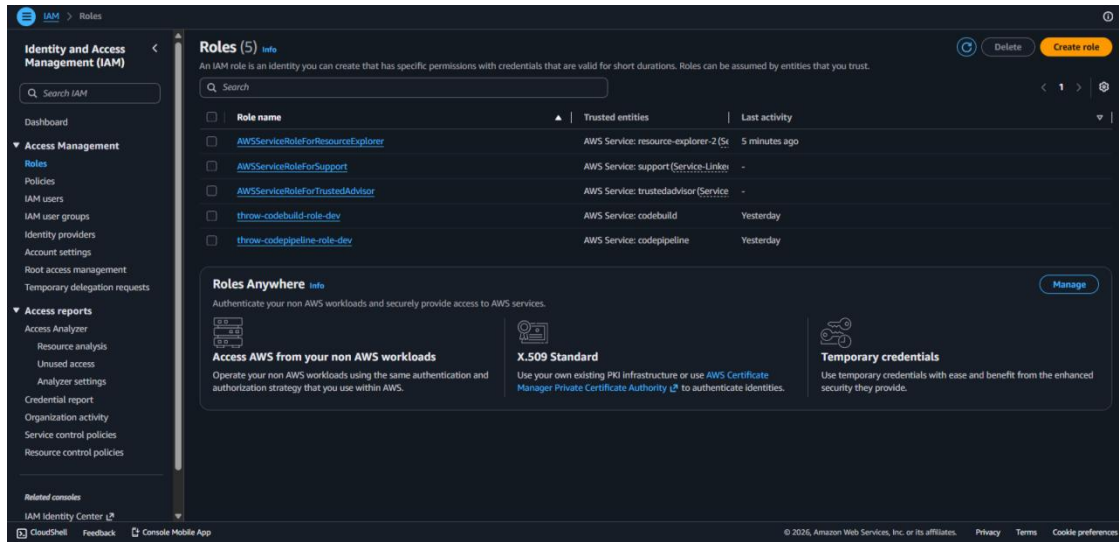


FIGURE 3 IAM Roles: the two project-specific roles alongside the account's default service-linked roles.

4.3 Storage Layout

Four S3 buckets support the project, each created and managed by Terraform: the artifact store, the CloudFront access-log bucket, the pipeline pub-cache bucket, and the Terraform remote-state bucket.

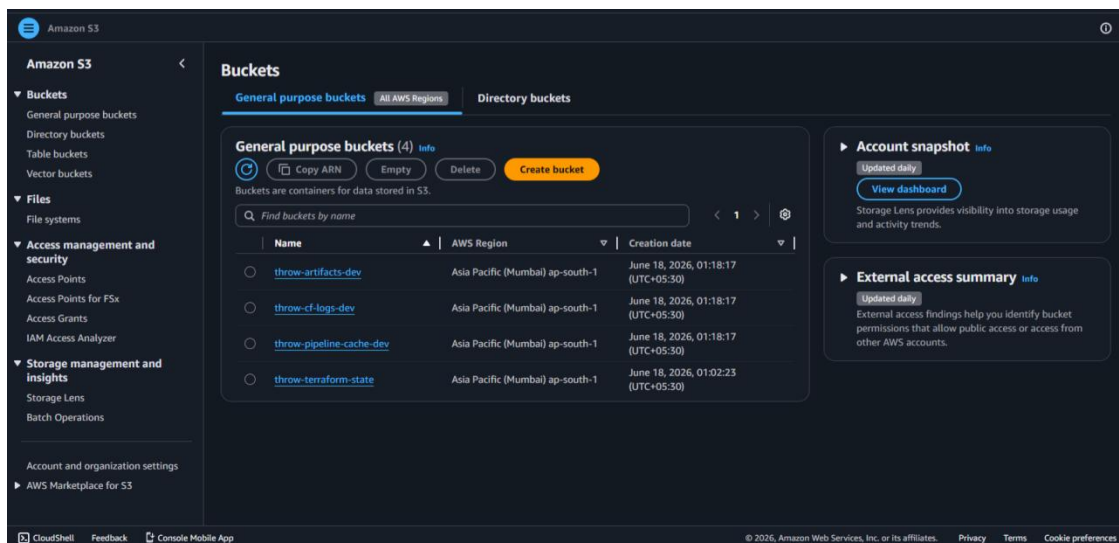


FIGURE 4 Amazon S3: `throw-artifacts-dev`, `throw-cf-logs-dev`, `throw-pipeline-cache-dev`, and `throw-terraform-state`.

4.4 Content Delivery

A single CloudFront distribution (E2AKDPXAL2GLD4) fronts the artifact bucket. It is the only path by which the built APKs are publicly reachable — the S3 origin itself has no public access (see Section 6.2).

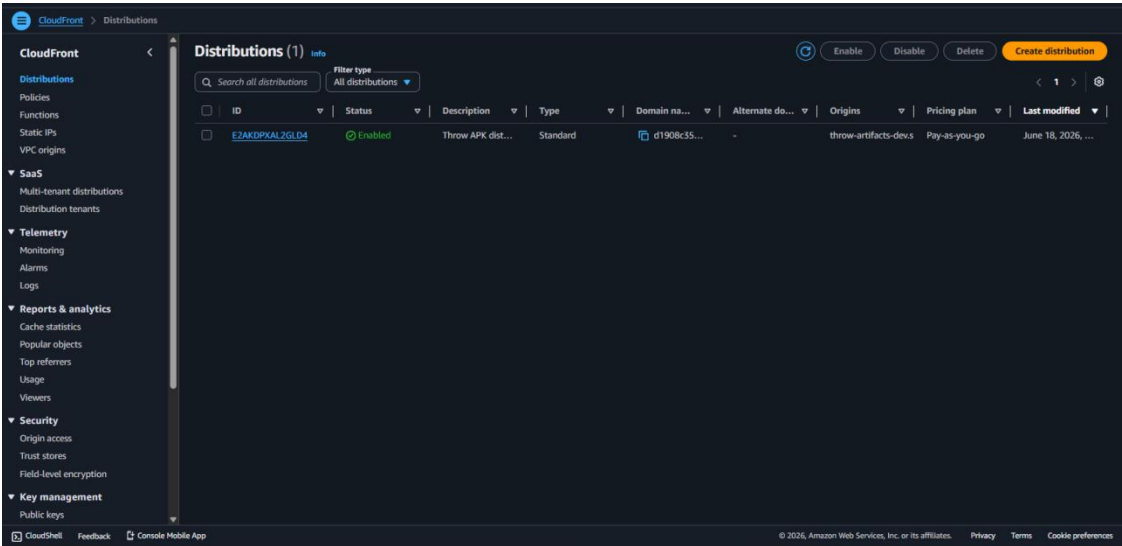


FIGURE 5 CloudFront distribution serving the Throw APK downloads, enabled and pointed at the artifact bucket.

4.5 Live Terraform Outputs

After a successful terraform apply, the configuration prints the following outputs directly to the terminal (account-identifying values, such as the GitHub connection ARN and SNS topic ARN, are redacted here):

```
Ubuntu
michu@DESKTOP-CSEJH1N:~$ cd throw-terraform
michu@DESKTOP-CSEJH1N:~/throw-terraform$ terraform output
apk_download_urls = {
  "throw_admin-main" = "https://d1908c35bz5y71.cloudfront.net/throw_admin-main/app-release.apk"
  "throw_delivery-main" = "https://d1908c35bz5y71.cloudfront.net/throw_delivery-main/app-release.apk"
  "throw_user-main" = "https://d1908c35bz5y71.cloudfront.net/throw_user-main/app-release.apk"
}
artifact_bucket_name = "throw-artifacts-dev"
cloudfront_domain = "https://d1908c35bz5y71.cloudfront.net"
cloudwatch_dashboard_url = "https://ap-south-1.console.aws.amazon.com/cloudwatch/home?region=ap-south-1#dashboards:name=throw-dashboard-dev"
github_connection_arn = "arn:aws:codestar-connections:ap-south-1: [REDACTED]"
github_connection_status = "AVAILABLE"
pipeline_urls = {
  "throw_admin-main" = "https://ap-south-1.console.aws.amazon.com/codesuite/codepipeline/pipelines/throw-throw_admin-main-dev/view"
  "throw_delivery-main" = "https://ap-south-1.console.aws.amazon.com/codesuite/codepipeline/pipelines/throw-throw_delivery-main-dev/view"
  "throw_user-main" = "https://ap-south-1.console.aws.amazon.com/codesuite/codepipeline/pipelines/throw-throw_user-main-dev/view"
}
sns_alerts_topic_arn = "arn:aws:sns:ap-south-1: [REDACTED]"
michu@DESKTOP-CSEJH1N:~/throw-terraform$
```

FIGURE 6 terraform output: CloudFront APK download URLs, artifact bucket name, and pipeline console links.

Output	Value
--------	-------

Output	Value
cloudfront_domain	https://d1908c35bz5y71.cloudfront.net
artifact_bucket_name	throw-artifacts-dev
apk_download_urls.throw_admin-main	.../throw_admin-main/app-release.apk
apk_download_urls.throw_delivery-main	.../throw_delivery-main/app-release.apk
apk_download_urls.throw_user-main	.../throw_user-main/app-release.apk
github_connection_status	AVAILABLE
cloudwatch_dashboard_url	AWS Console → CloudWatch → Dashboards → throw-dashboard-dev

CI/CD Pipeline Detail

5.1 Pipeline Stages

Each of the three pipelines (throw-throw_user-main-dev, throw-throw_delivery-main-dev, throw-throw_admin-main-dev) follows the same three stages, and all three are currently green end-to-end:

- **Source** — a CodeStar Connection pulls the latest commit from the app's GitHub repo on every push to main.
- **Build** — CodeBuild runs the builds spec defined in codebuild.tf (detailed below).
- **Deploy / Upload** — the built, signed APK artifact is uploaded to the project's path inside the S3 artifact bucket.

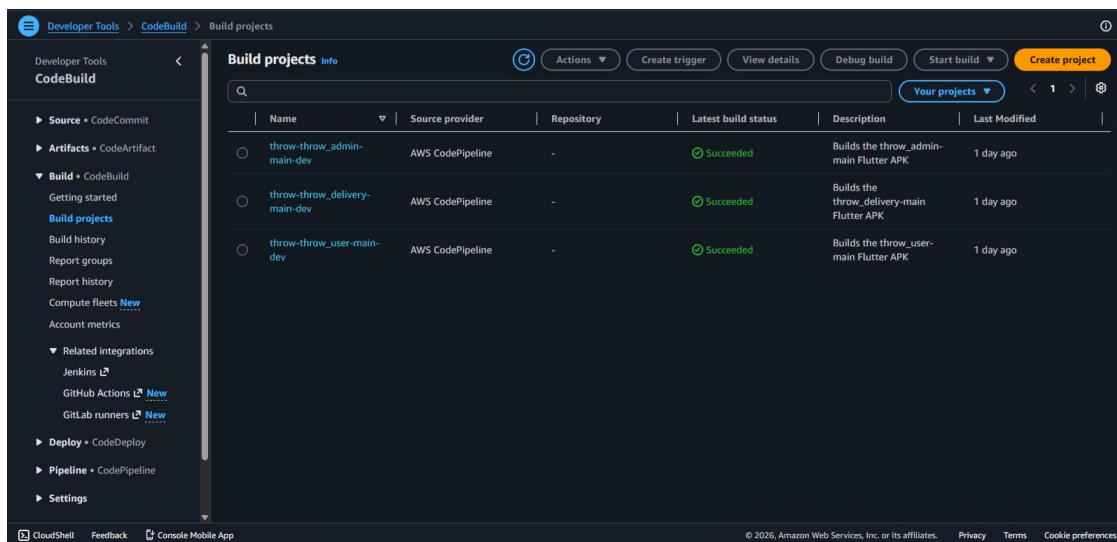


FIGURE 7 CodeBuild: all three projects (admin, delivery, user) with their most recent build Succeeded.

5.2 CodeBuild Phases

The builds spec is generated inline by Terraform via `yamlencode()` inside `codebuild.tf`. It has four phases:

Install Phase

- Downloads and installs the Android command-line tools (commandlinetools-linux, version 11076708)
- Accepts SDK licenses non-interactively and installs platform-tools, build-tools;35.0.0, and platforms;android-35
- Clones the Flutter SDK at a pinned version (branch 3.44.2) into `/opt/flutter`
- Points Flutter at the installed Android SDK and runs `flutter precache --android`

Pre-build Phase

- Writes the app's Firebase config from Secrets Manager into `lib/firebase_options.dart` and `android/app/google-services.json`
- Writes the release-signing keystore from Secrets Manager to disk and generates `key.properties`
- Runs `flutter pub get` to resolve Dart/Flutter dependencies
- Runs `dart run build_runner build --delete-conflicting-outputs` for apps using code generation (Freezed / `json_serializable`)

Build Phase

- Runs `flutter build apk --release`, which invokes Gradle's `assembleRelease` task and signs the APK with the release keystore

Post-build Phase

- Uploads the resulting `build/app/outputs/flutter-apk/app-release.apk` to `s3://throw-artifacts-dev/<app-name>/app-release.apk`

5.3 Secrets Injected at Build Time

Environment Variable	Source	Written To
GOOGLE_SERVICES_JSON	Secrets Manager: throw/<app>/google-services	android/app/google-services.json
FIREBASE_OPTIONS	Secrets Manager: throw/<app>/firebase-options	lib/firebase_options.dart
KEYSTORE / KEYSTORE_PASSWORD	Secrets Manager: throw/keystore, throw/keystore-password	android/app/throw-release.keystore + key.properties

These secrets are referenced declaratively in the `buildspec's env.secrets-manager` block, so CodeBuild fetches and injects them automatically — they are never written to the Terraform state or logged in plaintext.

5.4 Build History

The screenshot below, captured from the `throw_user-main` CodeBuild project, shows the exact before/after of the memory fix described in Incident 5 (Section 7): build #23 failed after 12 minutes 43 seconds when the Gradle/Kotlin daemon was killed by an out-of-memory condition, and build #24 — run immediately after the CodeBuild compute type was upgraded — succeeded in 11 minutes 49 seconds.

Developer Tools

CodeBuild

Source • CodeCommit

Artifacts • CodeArtifact

Build • CodeBuild

Getting started

Build projects

Build project

Settings

Build history

Report groups

Report history

Compute fleets **New**

Account metrics

Related integrations

Jenkins

GitHub Actions **New**

GitLab runners **New**

Deploy • CodeDeploy

Pipeline • CodePipeline

throw-throw_user-main-dev

Actions

Create trigger

Edit

Clone

Debug build

Start build with overrides

Start build

Configuration

Source provider
AWS CodePipeline

Primary repository

Artifacts upload location

Service role
arn:aws:iam::[redacted]:role/[redacted]

Public builds
Disabled

Build history

Batch history

Project details

Build triggers

Metrics

Debug sessions

Build history

Stop build

View artifacts

View logs

Delete builds

Retry build

Debug build

< 1 2 >

⚙️

☐	Build run	Status	Build number	Source version	Submitter	Duration	Completed
☐	throw-throw_user-main-dev [redacted]	✔️ Succeeded	24	arnaws33:throw-artifacts-dev/throw-throw_user-main/source_out/zxG6WVo	codepipeline/throw-throw_user-main-dev	11 minutes 49 seconds	1 day ago
☐	throw-throw_user-main-dev [redacted]	❌ Failed	23	arnaws33:throw-artifacts-dev/throw-throw_user-main/source_out/HtdOUT	codepipeline/throw-throw_user-main-dev	12 minutes 43 seconds	1 day ago

FIGURE 8 *throw_user-main build history: build #24 Succeeded vs. build #23 Failed (the OOM crash described in Incident 5).*

Security Decisions

6.1 No Credentials in Source Control

Firebase service credentials and the Android release-signing keystore were identified in (or required by) the original codebase, removed from / kept out of all three repositories, and migrated to AWS Secrets Manager. The project's .gitignore was hardened to explicitly exclude terraform.tfvars, the .terraform/ directory, all *.tfstate files, override files, and key/PEM/keystore files.

6.2 Private S3, OAC-Gated Access

The artifact S3 bucket is not publicly accessible. A bucket policy grants s3:GetObject only to the cloudfront.amazonaws.com service principal, scoped with a Condition restricting AWS:SourceArn to this project's specific CloudFront distribution ARN (arn:aws:cloudfront::<account-id>:distribution/E2AKDPXAL2GLD4). This is the standard Origin Access Control (OAC) pattern, which replaced the older, less secure Origin Access Identity (OAI) approach.

6.3 Least-Privilege IAM

Separate IAM roles exist for CodePipeline and CodeBuild. The CodeBuild role's policy (shown below, with the AWS account ID redacted) is scoped to exactly three things: writing to its own CloudWatch log group, reading and writing only the project's two S3 buckets, and reading only secrets under the throw/* path in Secrets Manager — it has no broader account access.

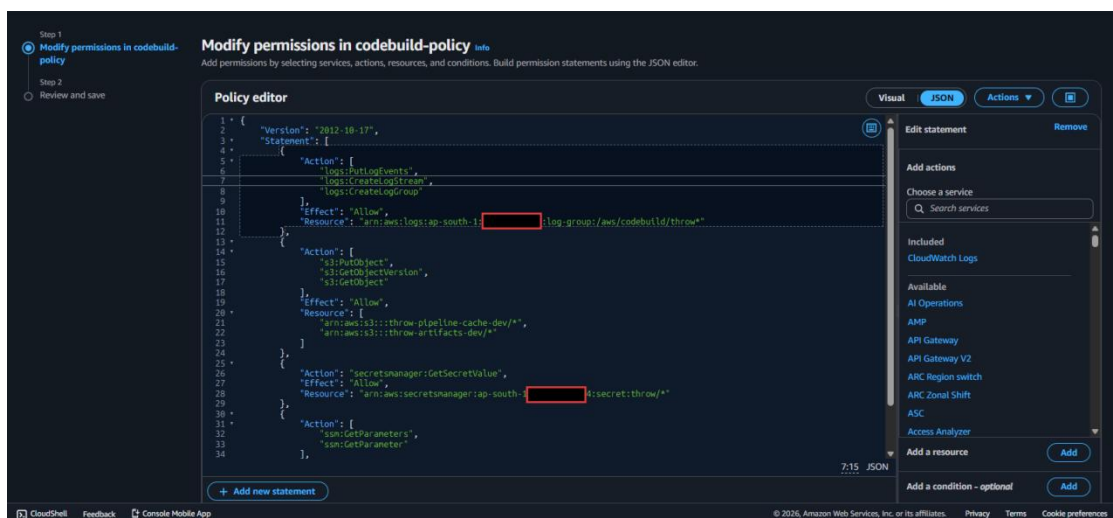


FIGURE 9 *throw-codebuild-policy: least-privilege JSON policy scoping CodeBuild to its own logs, buckets, and secrets.*

All Terraform-managed resources are also consistently tagged (Project, Environment, ManagedBy), which makes it straightforward to audit exactly what this project owns inside the AWS account.

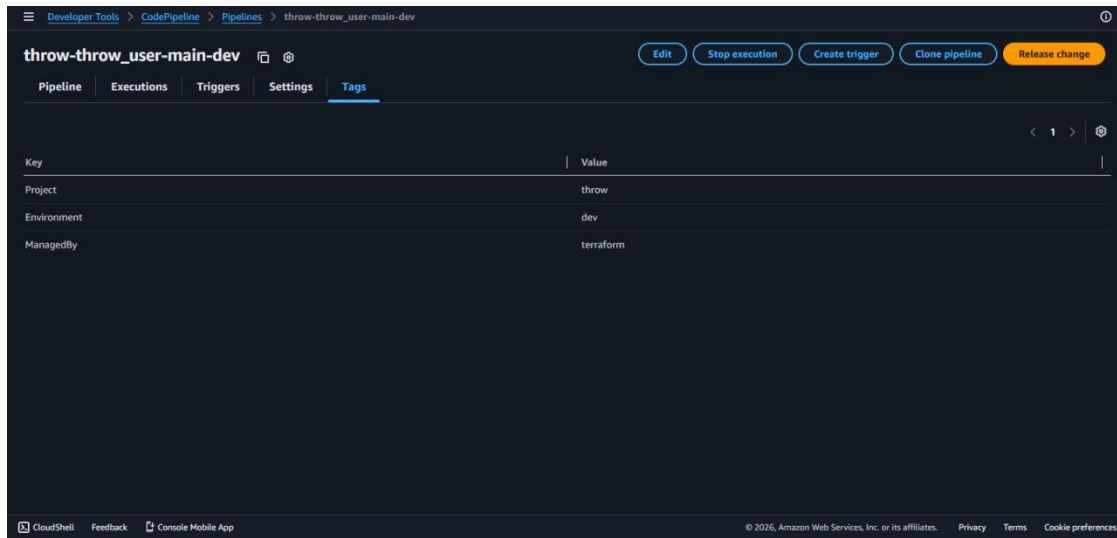


FIGURE 10 Resource tags applied consistently across every Terraform-managed resource in the project.

6.4 Git Credential Hygiene

During the project, a GitHub Personal Access Token expired mid-session, surfacing as an “Invalid username or token” push failure. This was resolved by generating a new classic PAT scoped to the repo permission and enabling git's `credential.helper` store so the token persists locally without re-entry on every push.

Incident Log — Build Debugging Timeline

Getting from “applied Terraform” to “working, installable, signed APK on all three apps” required working through a sequence of distinct failures. This section documents them in order, since the troubleshooting process is itself a core part of the DevOps work completed. All six incidents below are now fully resolved.

Incident 1 — CloudFront Returned Access Denied

Symptom: All three CloudFront URLs returned an XML Access Denied error on first request.

Cause: The S3 bucket policy had not yet granted the CloudFront distribution's Origin Access Control permission to read objects.

Fix: Applied an explicit bucket policy via `aws s3api put-bucket-policy` granting `s3:GetObject` to the `cloudfront.amazonaws.com` principal, conditioned on the specific distribution ARN.

STATUS

Resolved

Incident 2 — Downloaded File Was the Wrong Artifact

Symptom: Once CloudFront access was fixed, the downloaded “apk” files (named latest) failed to install on Android with a “Parsing failed” error.

Cause: Diagnosed using the `file` command, which revealed the downloaded object was a Zip archive — specifically, CodePipeline's default Deploy stage was uploading the entire source artifact bundle (the whole repo, zipped), not the compiled APK. At this point, no real flutter build apk had ever actually executed; the `buildspec`'s build phase still contained an `echo "build"` placeholder.

Fix: Rewrote `codebuild.tf` with a real `buildspec`, and changed the Terraform artifacts block to capture only `build/app/outputs/flutter-apk/app-release.apk`, with an explicit `aws s3 cp` upload in the `post_build` phase.

STATUS

Resolved

Incident 3 — Android SDK Not Found

Symptom: `flutter build apk --release` failed immediately with “no Android SDK found”, despite flutter config --android-sdk being run beforehand.

Cause: Diagnosed by running `find /` for `adb` and `sdkmanager` inside the build container, which returned nothing. The `aws/codebuild/standard:7.0` image does not ship with the Android SDK pre-installed at all — earlier assumptions about a default path (`/usr/local/android-sdk`) were incorrect.

Fix: Added explicit install-phase steps to download Android cmdline-tools directly from Google (`commandlinetools-linux-11076708_latest.zip`), unpack them to `/opt/android-sdk`, accept SDK licenses, and install the platform-tools, build-tools, and platform packages before invoking Flutter.

STATUS

Resolved

Incident 4 — Missing `firebase_options.dart`

Symptom: With the SDK now present, the build progressed much further but failed at the Dart compile step: “Error when reading 'lib/firebase_options.dart': No such file or directory.”

Cause: `firebase_options.dart` is a generated file required by `main.dart` at compile time. It existed locally on the developer's machine but had never been committed (correctly, since it is credential-bearing) and was not yet provisioned by the pipeline.

Fix: Located the file locally for the user and delivery apps, uploaded its contents as a new Secrets Manager secret (`throw/<app>/firebase-options`) per app, and added a `pre_build` step writing it to `lib/firebase_options.dart` from the `FIREBASE_OPTIONS` environment variable before `flutter pub get` runs. (The admin app has no such file, as it remained on the default Flutter scaffold at this stage.)

STATUS

Resolved

Incident 5 — Gradle/Kotlin Daemon Crash (JVM OOM)

Symptom: With all source files in place, the `throw_user-main` and `throw_delivery-main` builds progressed through dependency resolution and code generation successfully, then failed during the Gradle `assembleRelease` task with: “Gradle build daemon disappeared unexpectedly” and a Kotlin daemon `DaemonCrashedException / EOFException`, after running for several minutes and installing Android SDK Platforms 33/34/36 and the NDK.

Cause: This is a classic out-of-memory signature for Android/Kotlin Gradle builds — the JVM-based Gradle and Kotlin compiler daemons were killed by the kernel under memory pressure. The `throw_admin-main` app (a minimal scaffold) succeeded on the exact same instance class, while the larger apps — with more plugins (Firebase, OSM mapping, camera, geolocation) — exhausted the available memory on the `BUILD_GENERAL1_MEDIUM` CodeBuild compute type (3 GB RAM).

Fix: Upgraded the CodeBuild `compute_type` from `BUILD_GENERAL1_MEDIUM` to `BUILD_GENERAL1_LARGE` (7 GB RAM) for all three projects via Terraform, then re-released the user and delivery pipelines.

STATUS

Resolved and confirmed — build #24 (`throw_user-main`) succeeded in 11 minutes 49 seconds on the upgraded compute type, immediately following the failed build #23 (12 minutes 43 seconds) that exhibited the OOM crash. See Figure 8 in Section 5.4.

Incident 6 — Git Push Authentication Failure

Symptom: A routine git push to `throw-infra` was rejected with “Invalid username or token. Password authentication is not supported for Git operations.”

Cause: The previously generated GitHub Personal Access Token had expired or been revoked.

Fix: Generated a new classic PAT with repo scope and a 90-day expiry, used it once to authenticate the pull/push, and enabled git config `--global credential.helper store` to cache it for future operations.

STATUS

Resolved

7.1 Status Snapshot — Before and After

The figure below was captured directly from the AWS CodePipeline console immediately after Incident 4 was resolved, but before the memory fix (Incident 5) was applied. The admin app's pipeline had already succeeded; the user and delivery pipelines were still failing at the build stage for the reason documented above.

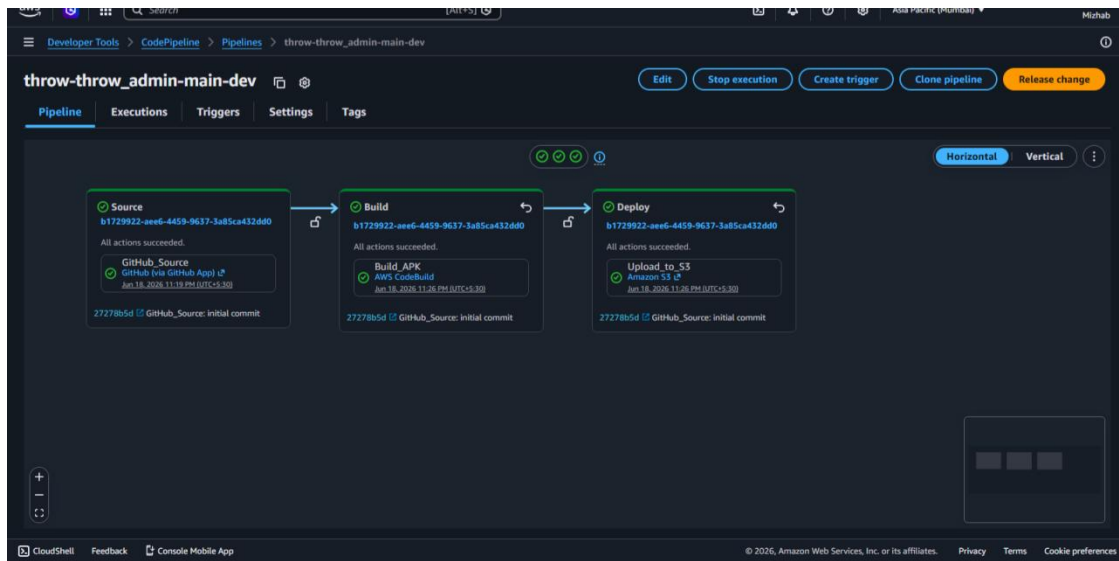


FIGURE 11 BEFORE: *throw_admin-main* already succeeded end-to-end while the memory issue was still being diagnosed (June 18, 2026).

After upgrading the CodeBuild compute type and re-releasing both affected pipelines, all three reached the same fully green state, each completing Source → Build → Deploy successfully:

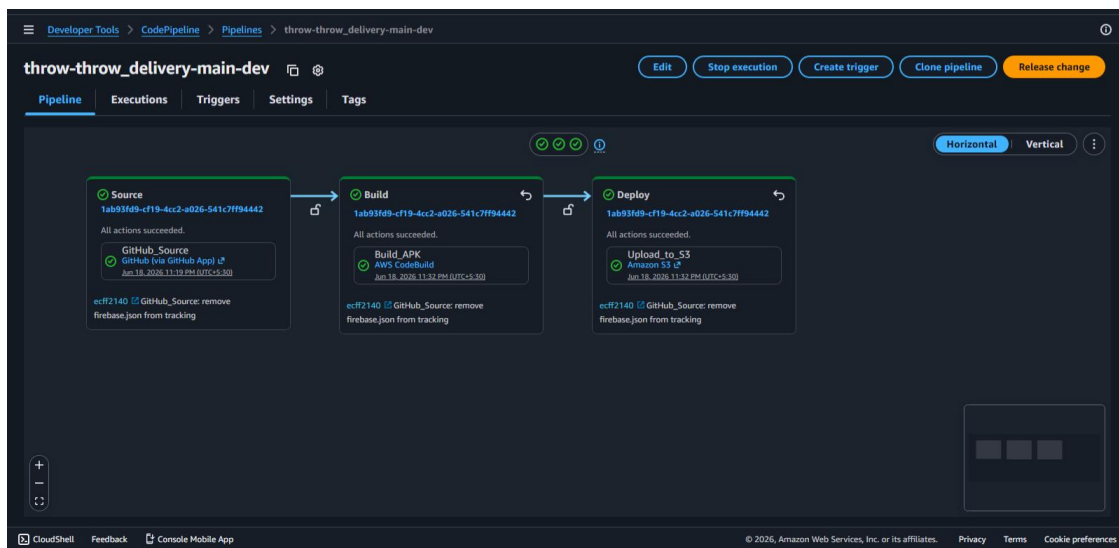


FIGURE 12 AFTER: *throw_delivery-main* succeeded end-to-end following the `BUILD_GENERAL1_LARGE` upgrade.

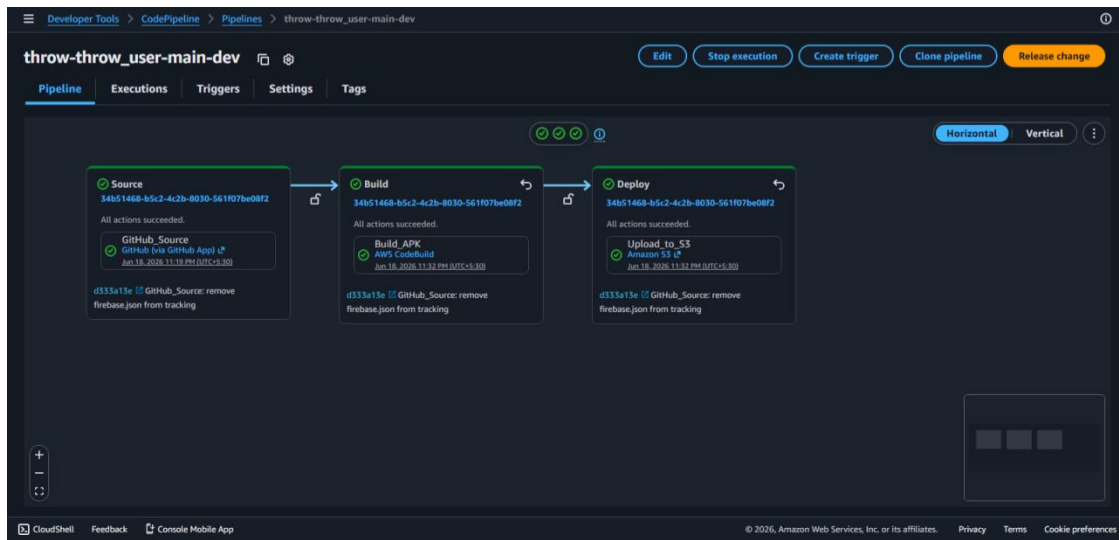


FIGURE 13 AFTER: *throw_user-main* succeeded end-to-end following the same fix.

The master pipeline dashboard (Figure 1, Section 1) confirms the final state: all three pipelines green, with each app producing a downloadable, installable APK via CloudFront.

How to Deploy / Reproduce

This section walks through reproducing this exact setup from a clean AWS account, in the order it was actually done, including the expected output at each step and what to check if something doesn't match. Steps that require the AWS Console (rather than the CLI or Terraform) are explicitly called out, since those are the only parts that can't be scripted.

8.1 Prerequisites

- Terraform v1.5 or newer (terraform -version to check)
- AWS CLI v2, configured with an IAM user/role that has administrator-level access for the initial setup (aws sts get-caller-identity should return your account ID without error)
- A GitHub account, with a personal access token (classic, repo scope) ready for CodeStar Connections authorization
- Flutter SDK installed locally (only needed to extract google-services.json / firebase_options.dart from your own Firebase project — the pipeline itself installs its own copy of Flutter inside CodeBuild)
- The three Flutter app repositories (throw_user-main, throw_delivery-main, throw_admin-main) already pushed to GitHub as private repos, with Firebase config files removed per Section 6.1

8.2 Step-by-Step Deployment

Step 1 — Create the Remote State Bucket

Terraform needs somewhere to store its state file before it can manage anything else, so this one bucket is created manually, once, outside of Terraform itself:

```
aws s3 mb s3://throw-terraform-state --region ap-south-1

aws s3api put-bucket-versioning \
  --bucket throw-terraform-state \
  --versioning-configuration Status=Enabled
```

Expected output: no errors; the second command returns nothing on success. Versioning is enabled so that a corrupted or accidentally-deleted state file can be recovered from a previous version.

Step 2 — Configure Variables

```
cd throw-infra
cp terraform.tfvars.example terraform.tfvars
# Edit terraform.tfvars: set your GitHub username and a notification email
```


terraform.tfvars is git-ignored deliberately — it is the one file allowed to contain account-specific (though not secret) values like your GitHub username and notification email address.

Step 3 — Initialize and Review the Plan

```
terraform init
terraform plan
```

terraform init downloads the AWS provider and configures the S3 backend from Step 1. terraform plan should report roughly 35–40 resources to add and zero to change or destroy on a first run. Review the plan output before proceeding — in particular, confirm the for_each blocks show three instances each (one per app) for the CodeBuild project, CodePipeline, and the Firebase-related secrets.

Step 4 — Apply

```
terraform apply
```

Type yes when prompted. This step typically takes 2–4 minutes — most of that time is CloudFront distribution creation, which is the slowest resource to provision. On success, Terraform prints the full outputs block (CloudFront domain, APK download URLs, pipeline console links).

Step 5 — Authorize the GitHub Connection (Manual, One-Time, Console Only)

CONSOLE-ONLY STEP

This is the one step that genuinely cannot be scripted, because AWS requires interactive OAuth consent for CodeStar Connections.

1. Open the AWS Console → Developer Tools → CodePipeline → Settings → Connections.
2. Find the connection created by Terraform (status will read Pending).
3. Select it → click Update pending connection.
4. Authorize via the GitHub OAuth prompt, granting access to the three app repositories.
5. Confirm the connection status changes to Available (this is also surfaced as the github_connection_status Terraform output — it should read "AVAILABLE").

COMMON FIRST-RUN BLOCKER

If this step is skipped, every pipeline's Source stage will fail immediately with a connection error — this is the most common first-run blocker.

Step 6 — Populate Firebase Secrets

With the infrastructure live, push the real Firebase config values (extracted from your own Firebase project) into the empty Secrets Manager entries Terraform created:

```
aws secretsmanager put-secret-value \
  --secret-id throw/throw_user-main/google-services \
  --secret-string file://throw_user-main/android/app/google-services.json \
  --region ap-south-1

aws secretsmanager put-secret-value \
  --secret-id throw/throw_user-main/firebase-options \
  --secret-string file://throw_user-main/lib/firebase_options.dart \
  --region ap-south-1

# Repeat both commands for throw_delivery-main
# (throw_admin-main only needs this once it gains real Firebase usage)
```

To verify the secrets were written correctly without printing the sensitive value to your terminal, check only the metadata:

```
aws secretsmanager describe-secret --secret-id throw/throw_user-main/google-services
```

Step 7 — Populate the Release-Signing Keystore

The signing keystore and its password are stored the same way, as binary-safe base64 and a plain string respectively:

```
aws secretsmanager put-secret-value \
  --secret-id throw/keystore \
  --secret-string fileb://throw-release.keystore \
  --region ap-south-1

aws secretsmanager put-secret-value \
  --secret-id throw/keystore-password \
  --secret-string 'your-keystore-password' \
  --region ap-south-1
```

Once populated, all secrets relevant to the project are visible (names and metadata only — never values) in the Secrets Manager console:

Secret name	Description	Last retrieved (UTC)	Created on (UTC)
throw/keystore-password	-	June 18, 2026	June 18, 2026 at 14:09:39
throw/keystore	-	June 18, 2026	June 18, 2026 at 14:09:37
throw/throw_admin-main/google-services	google-services.json for throw_admin-main Firebase project	June 18, 2026	June 17, 2026 at 19:48:17
throw/throw_admin-main/firebase-options	firebase_options.dart for throw_admin-main	June 18, 2026	June 17, 2026 at 19:48:16
throw/throw_user-main/firebase-options	firebase_options.dart for throw_user-main	June 18, 2026	June 17, 2026 at 19:48:16
throw/throw_delivery-main/google-services	google-services.json for throw_delivery-main Firebase project	June 18, 2026	June 17, 2026 at 19:48:16
throw/throw_admin-main/sha1-fingerprint	Android SHA-1 fingerprint for throw_admin-main	-	June 17, 2026 at 19:48:16
throw/throw_delivery-main/firebase-options	firebase_options.dart for throw_delivery-main	June 18, 2026	June 17, 2026 at 19:48:16
throw/throw_user-main/sha1-fingerprint	Android SHA-1 fingerprint for throw_user-main	-	June 17, 2026 at 19:48:16
throw/throw_delivery-main/sha1-fingerprint	Android SHA-1 fingerprint for throw_delivery-main	-	June 17, 2026 at 19:48:16

FIGURE 14 AWS Secrets Manager: Firebase config, SHA-1 fingerprints, and the release keystore, one entry per app.

Step 8 — Trigger the First Real Build

Push any change to main on any of the three Flutter repos — CodePipeline picks it up automatically via the webhook configured through the GitHub connection. Alternatively, to re-run the latest commit without making a new commit, use Release change in the CodePipeline console for that pipeline.

```
git add .
git commit -m "trigger pipeline"
git push origin main
```

Expect the full Source → Build → Deploy cycle to take roughly 10–15 minutes on first run for throw_user-main and throw_delivery-main (Android SDK download/install plus a full Gradle build), and a few minutes less for throw_admin-main.

Step 9 — Watch the Build, and What to Check if It Fails

Use Build history on the relevant CodeBuild project to follow progress, and View logs on any run to see the live buildspec output.

If the Build Fails At...	Most Likely Cause	Where to Look
Install phase	Android SDK / cmdline-tools download or license step	Section 7, Incident 3
Pre-build phase, Dart compile error	Missing firebase_options.dart for that app	Section 7, Incident 4
Build phase, Gradle daemon crash	Out-of-memory — compute type too small	Section 7, Incident 5
Deploy stage / Upload_to_S3	IAM permissions on the artifact	Section 6.3 (least-privilege policy)

If the Build Fails At...	Most Likely Cause	Where to Look
	bucket	
Source stage, every run	GitHub connection not yet authorized	Step 5 above

A successful run looks like this — all three stages green, with the Build_APK action under AWS CodeBuild and Upload_to_S3 under Amazon S3 both showing a checkmark:

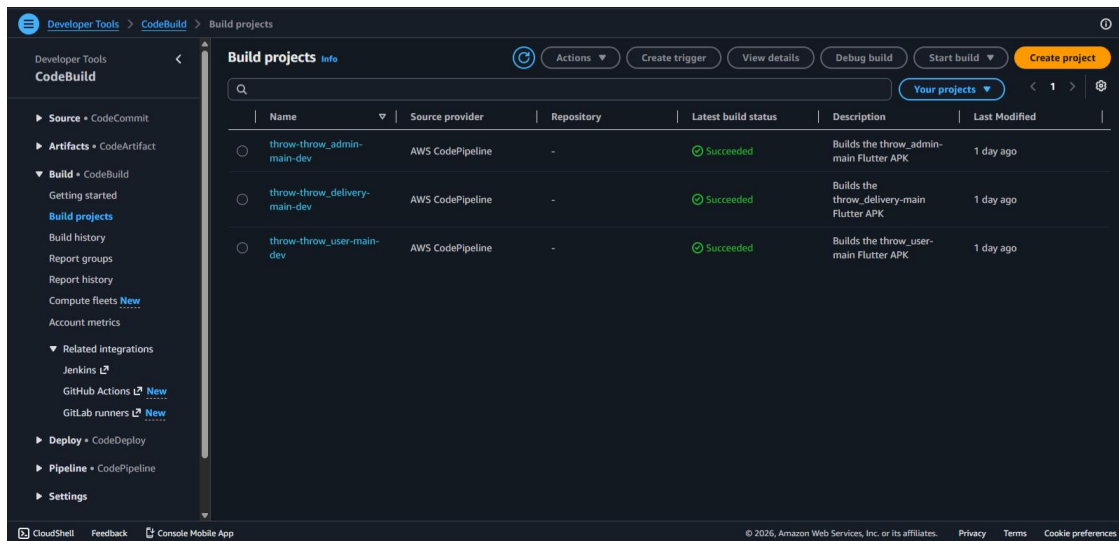


FIGURE 15 All three CodeBuild projects with their latest build Succeeded — the expected end state after Step 9.

Step 10 — Retrieve the Download Links

```
terraform output
```

The `apk_download_urls` output map gives the direct CloudFront URL for each app's latest successful build. These URLs are stable — each new successful pipeline run overwrites the same S3 key, so the same link always serves the most recent build (see Section 9 for installing the APK from this URL).

Step 11 — Confirm Monitoring Is Live

Check that CloudWatch is receiving logs and that all alarms show OK (not In alarm or Insufficient data):

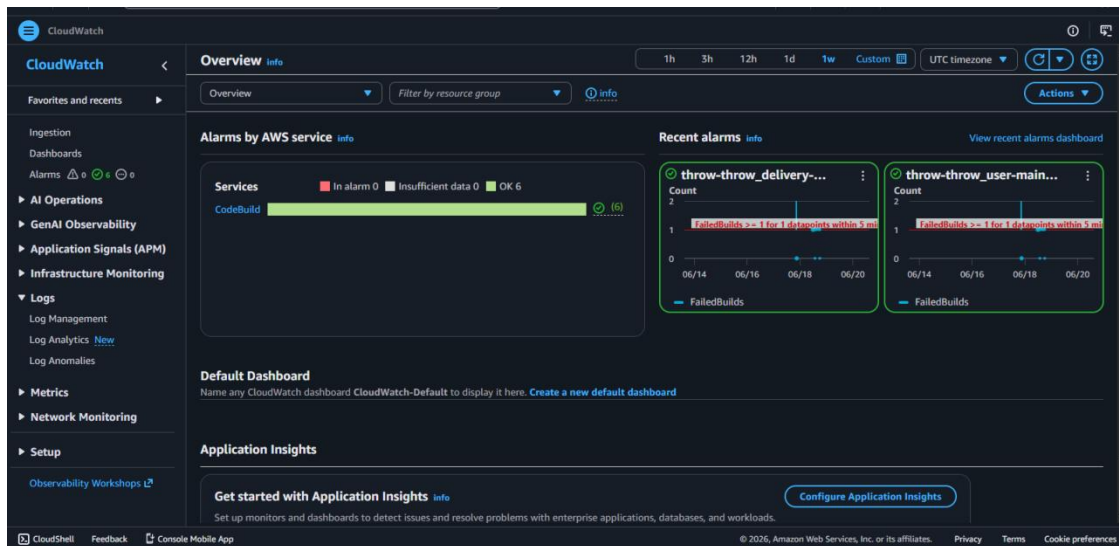


FIGURE 16 CloudWatch: all 6 build-failure alarms (2 per app × 3 apps... consolidated to 6 total) showing OK, with the FailedBuilds history visibly recovering after the Incident 5 fix.

Step 12 — Tear Down Between Sessions (Cost Control)

```
terraform destroy
```

Type yes when prompted. Running destroy between working sessions reduces AWS spend to effectively zero, since none of the provisioned resources (S3, CloudFront, CodeBuild, CodePipeline) incur meaningful idle cost once removed; only the billing alarm/SNS topic and Secrets Manager entries persist minor charges (a few cents per month) if left running long-term. Re-running terraform apply afterward recreates everything identically, since the entire infrastructure is defined as code.

NOTE

Destroying and re-applying will generate a new CloudFront domain and a new GitHub connection ARN, so Step 5 (GitHub authorization) and the download links in Step 10 will need to be redone / re-fetched after a fresh apply.

Installing the Built APK

Once a pipeline succeeds, the resulting signed APK can be installed on a real Android device or an emulator such as BlueStacks.

9.1 Direct Download on an Android Device

- Open the CloudFront URL for the desired app directly in the device's browser (e.g. https://d1908c35bz5y71.cloudfront.net/throw_admin-main/app-release.apk).
- Allow "Install unknown apps" for the browser if prompted (Settings → Apps → Special app access → Install unknown apps).
- Open the downloaded file and tap Install.

9.2 Sharing via WhatsApp

- Rename the downloaded file to include a .apk extension (e.g. throw_user_main.apk) — WhatsApp may reject files without a recognized extension.
- Send the file to your own chat (Saved Messages) from the desktop client.
- Open WhatsApp on the phone, download the attachment, and tap to install.

9.3 BlueStacks (Emulator)

Drag and drop the APK onto the BlueStacks window, or use My Games → Install APK.

LIMITATION

Firebase Authentication (Google Sign-In), GPS, and camera-dependent features may not function correctly in an emulator, since they depend on Google Play Services and real device sensors. BlueStacks is suitable for confirming the app launches and the UI renders correctly, but full functional testing should be done on a physical Android device.

Monitoring and Cost Control

10.1 CloudWatch

- Each CodeBuild project logs to its own CloudWatch Logs group (/aws/codebuild/throw, with a stream per app)
- Alarms are configured for build failures and unusually slow builds — currently all 6 alarms show OK
- A CloudWatch dashboard (throw-dashboard-dev) aggregates pipeline health at a glance

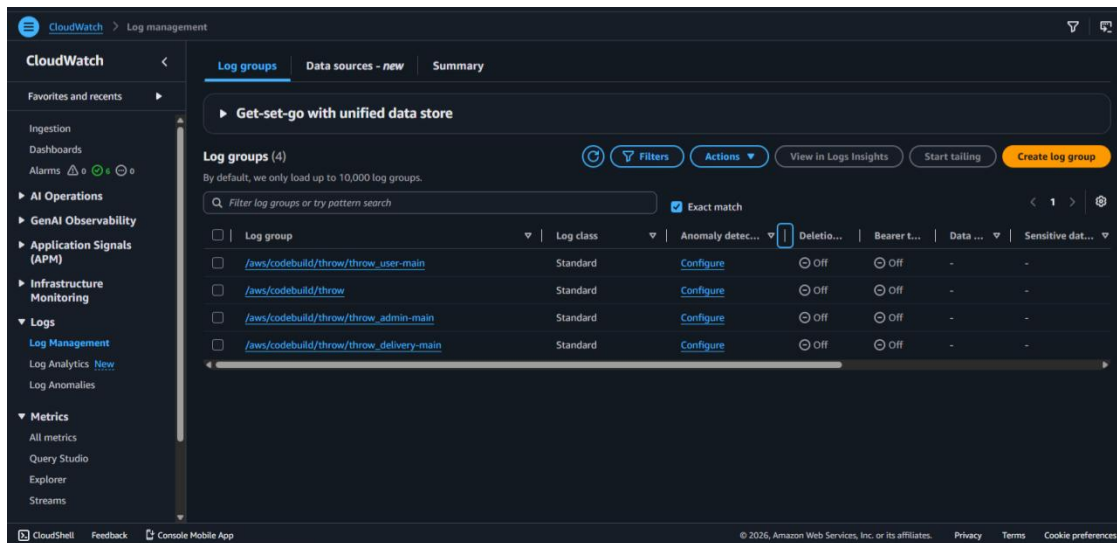


FIGURE 17 CloudWatch Log groups: one per app, plus a shared /aws/codebuild/throw group.

10.2 SNS Alerting

An SNS topic (throw-alerts-dev) fans out build-failure and billing-threshold notifications to email.

10.3 Billing Alarm

A billing alarm is configured in us-east-1 (required by AWS for billing metrics) at a fixed monthly USD threshold, so unexpected cost growth triggers an email alert rather than going unnoticed.

10.4 Estimated Monthly Cost

At portfolio/development scale — a handful of builds per day, low CDN egress, and minimal S3 storage — the estimated cost is approximately \$4–\$7/month while infrastructure is left running continuously, and close to \$0 when torn down between sessions via terraform destroy.

Repository Summary

Repository	Contents	Visibility
throw_user-main	Customer Flutter app	Private
throw_delivery-main	Courier Flutter app	Private
throw_admin-main	Admin Flutter app (pipeline live; app UI at scaffold stage)	Private
throw-infra	Terraform IaC for all AWS infrastructure	Private

Lessons Learned

- CodeBuild's standard Docker images do not include the Android SDK by default for Flutter/Android builds — it must be installed explicitly in the install phase.
- CodePipeline's default artifact passthrough uploads the entire stage input/output as a zip; producing a single binary artifact (like an APK) requires an explicit `artifacts.files` path matching the actual build output, plus a direct S3 upload if a stable, predictable object key is needed.
- Files required at compile time but deliberately excluded from version control (such as generated Firebase config or a signing keystore) must be reconstructed by the pipeline itself, sourced from a secret store — they don't appear “for free.”
- Android/Kotlin/Gradle builds are memory-hungry; CodeBuild's smallest compute tier is sufficient for minimal apps but insufficient for apps with several native-bridging plugins (Firebase, maps, camera, geolocation) — confirmed directly by comparing the failed and succeeded build durations in Figure 8.
- Terraform's `for_each` pattern scales cleanly across near-identical multi-app infrastructure, keeping the codebase maintainable as more apps are added.
- Origin Access Control (OAC) plus a scoped S3 bucket policy is the current best-practice way to let CloudFront read a private S3 bucket, replacing the older Origin Access Identity (OAI) approach.
- Screenshots and terminal output captured during a real debugging session often contain account IDs or ARNs in plaintext; reviewing each one before sharing or documenting (and redacting where needed) should be a standard step, not an afterthought.

Next Steps

- Build out the throw_admin-main app beyond its current default Flutter scaffold, now that its pipeline is fully proven.
- Add a staging environment (separate Terraform workspace or var.environment value) to test infrastructure changes before they affect the dev environment.
- Set up branch-based triggers (e.g. only main triggers a release build) once active feature-branch development begins.
- Revisit Gradle, Android Gradle Plugin, and Kotlin versions flagged as deprecated in the build logs to keep the apps on a supported toolchain long-term.
- Consider adding automated UI/integration tests as an earlier pipeline stage, so regressions are caught before the (comparatively slow) APK build runs.
- Review CodeBuild compute-type sizing periodically — BUILD_GENERAL1_LARGE resolved the immediate OOM issue, but as the apps grow, monitor build duration and memory headroom rather than assuming the current size is permanently sufficient.